

Decisiones Arquitectónicas (ADRs)

Entregable 4 – Práctica de Arquitectura y Diseño de Sistemas 2026

ID	Título	Estado	Fecha
ADR-001	Estrategia de organización de la lógica de negocio del backend	Aceptada	01/06/2026

1. Contexto

El sistema debe gestionar múltiples funcionalidades estrechamente relacionadas, tales como reservas de turnos, gestión de complejos, administración de equipamiento, pagos, penalizaciones y notificaciones.

Muchas operaciones del negocio involucran varios dominios simultáneamente. Por ejemplo, al registrar una reserva se debe verificar disponibilidad, validar penalizaciones, comprobar stock de equipamiento y actualizar el estado de la reserva.

Motivador 1 – Consistencia transaccional
Las operaciones de reserva y pago deben ejecutarse de forma atómica para cumplir RNF-10.

Motivador 2 – Complejidad del dominio
Funcionalidades como RF-07, RF-21, RF-29 y RF-30 involucran reglas de negocio que atraviesan distintos módulos del sistema.

Motivador 3 – Restricciones del proyecto
El sistema será desarrollado por un equipo reducido dentro del contexto académico de la materia.

2. Alternativas consideradas

Monolito simple sin modularización	Simplicidad inicial de implementación.	Genera alto acoplamiento entre funcionalidades y dificulta el mantenimiento.
Arquitectura de microservicios	Permite escalar dominios independientemente.	Introduce complejidad operativa y problemas de consistencia distribuida para operaciones críticas como reservas y pagos.
Monolito modular (elegida)	Permite separar responsabilidades por dominio manteniendo transacciones locales.	(esta fue la elegida)

3. Decisión tomada

Se decide: organizar el backend utilizando una arquitectura monolítica modular, separando la lógica en módulos de dominio tales como usuarios, reservas, complejos, equipamiento y administración.

Fundamentación

Razón 1: la arquitectura monolítica modular permite ejecutar dentro de una misma transacción operaciones críticas como reserva, validación de disponibilidad y actualización de equipamiento, satisfaciendo el motivador de consistencia transaccional.

Razón 2: la separación por dominios funcionales reduce el acoplamiento y facilita la evolución de funcionalidades complejas como pagos, penalizaciones y notificaciones, respondiendo al motivador de complejidad del dominio.

Razón 3: esta arquitectura presenta una complejidad operativa significativamente menor que una solución basada en microservicios, adecuándose a las restricciones de tiempo y experiencia del equipo.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
Facilita la implementación de reglas de negocio transaccionales.	El sistema completo debe desplegarse como una única unidad.
Favorece la organización del código por dominios funcionales.	El escalado debe realizarse sobre toda la aplicación y no sobre módulos individuales.
Reduce la complejidad de desarrollo y despliegue.	Un fallo en la aplicación puede afectar a todo el sistema.